

SIMATS ENGINEERING

Name: Keerthana S
Register Number: 192471047
Course Code: CSA0313
Course: Data Structures

CO2 – ASSESSMENT TOOL 3

1. INSERT A NEW NODE AT THE END OF A SINGLY LINKED LIST

Idea:

Instead of walking through the whole list every time we add a node, we can keep a separate pointer that always points to the last node. When a new node needs to be added, we simply attach it after the node the tail pointer refers to, and then move the tail pointer forward. This avoids traversing the list on every insertion.

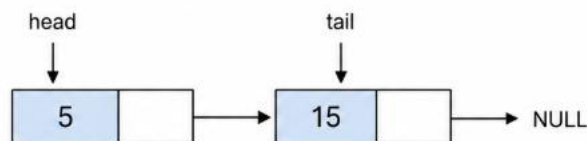
Steps:

- Create a new node using malloc().
- Read the data into temp->data.
- Set temp->next = NULL.
- If the list is empty, set both head and tail to temp.
- Otherwise, link tail->next to temp.
- Move tail forward to temp.

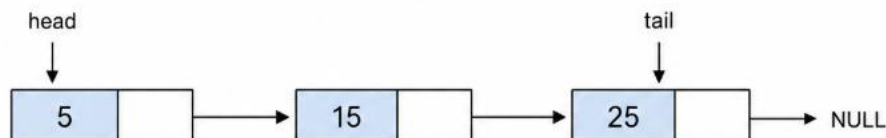
Simple diagram:

Simple diagram:

Before insertion:



Insert 25



Program:

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
int data;
struct node *next;
};
struct node *head = NULL;
struct node *tail = NULL;
void insert_end()
{
struct node *temp;
temp = (struct node*)malloc(sizeof(struct node));
printf("Enter node data: ");
scanf("%d", &temp->data);
temp->next = NULL;
if(head == NULL)
{
head = temp;
tail = temp;
}
else
{
tail->next = temp;
tail = temp;
}
}
void display()
{
struct node *ptr = head;
while(ptr != NULL)
{
printf("%d ", ptr->data);
ptr = ptr->next;
}
}
int main()
{
insert_end();
insert_end();
insert_end();
printf("Linked List: ");
```

```
display();  
return 0;  
}
```

Dry Run:

Assume the list contains 5 and 15, and tail points to node 15.

When 25 is inserted:

- tail currently points to node 15.
- Set 15->next = 25.
- Move tail to node 25.
- No traversal was needed.
- The final list becomes $5 \rightarrow 15 \rightarrow 25$.

Time Complexity:

$O(1)$ per insertion, since the tail pointer removes the need to traverse the list.

2. SEARCH FOR AN ELEMENT IN A SINGLY LINKED LIST

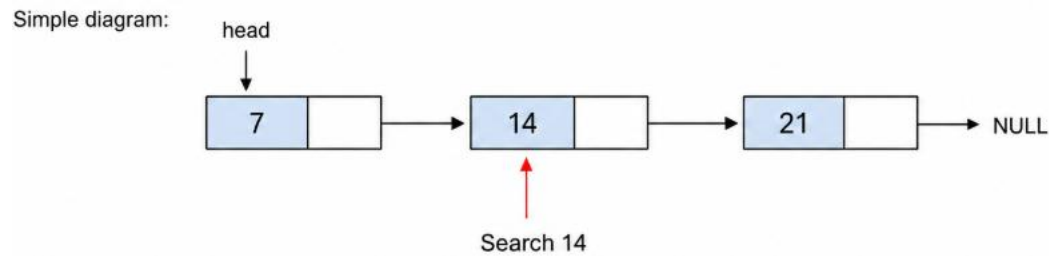
Idea:

Instead of using a loop, we can search the list recursively. Each recursive call checks one node: if it matches the key we stop and return its position, otherwise we call the function again on the next node with the position increased by one. The recursion ends naturally when we reach the end of the list.

Steps:

- Pass head, the search key, and starting position 1 to the function.
- If the current pointer is NULL, return -1 (not found).
- If the current node's data matches the key, return the current position.
- Otherwise, call the function again on `ptr->next` with position + 1.
- Use the returned value in main to print the result.

Simple diagram:



Program:

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
    int data;
    struct node *next;
};
struct node *head = NULL;
void insert(int value)
{
    struct node *temp, *ptr;
    temp = (struct node*)malloc(sizeof(struct node));
    temp->data = value;
    temp->next = NULL;
    if(head == NULL)
    {
        head = temp;
    }
    else
    {
        ptr = head;
        while(ptr->next != NULL)
        {
            ptr = ptr->next;
        }
        ptr->next = temp;
    }
}
int search_recursive(struct node *ptr, int key, int position)
{
    if(ptr == NULL)
```

```

{
return -1;
}
if(ptr->data == key)
{
return position;
}
return search_recursive(ptr->next, key, position + 1);
}
int main()
{
int key, pos;
insert(7);
insert(14);
insert(21);
printf("Enter element to search: ");
scanf("%d", &key);
pos = search_recursive(head, key, 1);
if(pos != -1)
{
printf("Element found at position %d", pos);
}
else
{
printf("Element not found");
}
return 0;
}

```

Dry Run:

Assume the list is: $7 \rightarrow 14 \rightarrow 21 \rightarrow \text{NULL}$. Search for 14.

- search_recursive(7, 14, 1): $7 \neq 14$, call search_recursive(14, 14, 2).
- search_recursive(14, 14, 2): $14 = 14$, return 2.
- The final result is Element found at position 2.

Time Complexity:

$O(n)$ for the comparisons, with an additional $O(n)$ call stack space used by the recursion.

3. DELETE A NODE FROM THE BEGINNING OF A DOUBLY LINKED LIST

Idea:

When removing the first node of a doubly linked list, the node right after it becomes the new first node. We move the head pointer to that node and clear its prev pointer so it no longer refers back to the deleted node.

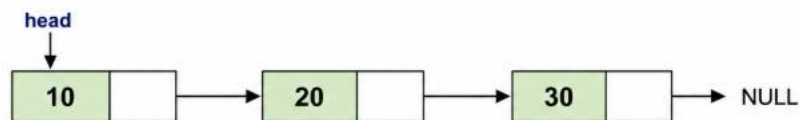
Steps:

- Check whether the list is empty.
- Store the first node in temp.
- Move head to head->next.
- If the new head is not NULL, set head->prev = NULL.
- Free the deleted node using free().

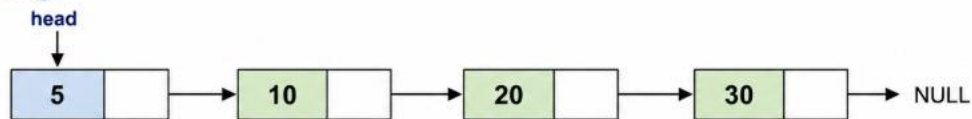
Simple diagram:

Simple diagram:

Before insertion:



Insert 5 at beginning:



Program:

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void delete_front()
{
    struct node *temp;
    if(head == NULL)
    {
```

```

printf("List is empty");
return;
}
temp = head;
head = head->next;
if(head != NULL)
{
head->prev = NULL;
}
free(temp);
}
void display()
{
struct node *temp = head;
while(temp != NULL)
{
printf("%d ", temp->data);
temp = temp->next;
}
}
int main()
{
struct node *a, *b, *c;
a = (struct node*)malloc(sizeof(struct node));
b = (struct node*)malloc(sizeof(struct node));
c = (struct node*)malloc(sizeof(struct node));
a->data = 5;
b->data = 15;
c->data = 25;
a->prev = NULL; a->next = b;
b->prev = a; b->next = c;
c->prev = b; c->next = NULL;
head = a;
printf("Before deletion: ");
display();
delete_front();
printf("\nAfter deletion: ");
display();
return 0;
}

```

Dry Run:

Initially: 5 ↔ 15 ↔ 25

- head points to 5.
- Store 5 in temp.
- Move head to 15.
- Set 15->prev = NULL.
- Free node 5.

Final list: 15 ↔ 25

Time Complexity:

$O(1)$

4. IMPLEMENT A QUEUE USING AN ARRAY

Idea:

A plain array-based queue wastes space once front moves ahead, because the freed slots at the beginning cannot be reused. A circular queue solves this by wrapping the rear and front indices back to the start of the array using the modulus operator, so all slots stay usable.

Steps:

- Initialize front = -1 and rear = -1.
- For insertion, check if $(\text{rear} + 1) \% \text{SIZE} == \text{front}$, which means the queue is full.
- If front == -1, set front = 0.
- Move rear forward with $\text{rear} = (\text{rear} + 1) \% \text{SIZE}$, and store the value.
- For deletion, remove the element at front, then move front forward with $\text{front} = (\text{front} + 1) \% \text{SIZE}$.
- If front == rear after deletion, reset both to -1 since the queue is empty.

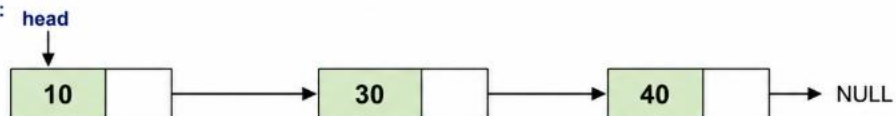
Simple diagram:

Simple diagram:

Before deletion (delete key = 20):



After deletion:



Program:

```
#include <stdio.h>
#define SIZE 5
int cqueue[SIZE];
int front = -1;
int rear = -1;
void enqueue(int value)
{
    if((rear + 1) % SIZE == front)
    {
        printf("Queue is full\n");
        return;
    }
    if(front == -1)
    {
        front = 0;
    }
    rear = (rear + 1) % SIZE;
    cqueue[rear] = value;
    printf("%d inserted\n", value);
}
void dequeue()
{
    if(front == -1)
    {
        printf("Queue is empty\n");
        return;
    }
    printf("%d deleted\n", cqueue[front]);
    if(front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front = (front + 1) % SIZE;
    }
}
void display()
{
    int i;
    if(front == -1)
```

```

{
printf("Queue is empty\n");
return;
}
printf("Queue: ");
i = front;
while(1)
{
printf("%d ", cqueue[i]);
if(i == rear)
{
break;
}
i = (i + 1) % SIZE;
}
printf("\n");
}
int main()
{
enqueue(11);
enqueue(22);
enqueue(33);
display();
dequeue();
display();
return 0;
}

```

Dry Run:

Initially: front = -1, rear = -1.

Insert 11: front = 0, rear = 0 → queue: 11

Insert 22: rear = 1 → queue: 11 22

Insert 33: rear = 2 → queue: 11 22 33

Dequeue removes 11 (at front, index 0), front moves to index 1.

Final queue: 22 33

Time Complexity:

- Enqueue: $O(1)$
- Dequeue: $O(1)$
- Display: $O(n)$